

Human vs. Zombie

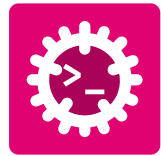
Laboratory - Advanced Programming



HES-SO Valais//Wallis
Systems Engineering • Infotronics
Advanced Programming • Silvan Zahno
Fall Semester 2026 • I3

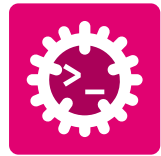
Silvan Zahno, Rémy Borgeat

05.08.2026 • v1.0 • final



Contents

1	Goal	3
1.1	The Game	3
1.2	Project Setup	3
2	The Position Struct	4
3	The Human Struct	5
3.1	How Movement Works (Pre-implemented)	5
4	The Zombie Struct	6
4.1	How Movement Works (Pre-implemented)	6
5	The World Struct	7
5.1	Pre-implemented Methods	7
5.2	Final Verification	7
6	Checkout Part 1	8
7	The Entity Trait	9
7.1	Step 1: Define the Trait	9
7.2	Step 2: Implement Entity for Human	9
7.3	Step 3: Implement Entity for Zombie	10
7.4	Updating find_free_neighbor	10
8	World Refactoring	11
8.1	The Struct	11
8.2	Adding Entities	11
8.3	Movement	11
8.4	Infection	12
8.5	Decay	12
8.6	Display	13
8.7	Verification	13
9	Checkout Part 2	14
	Glossary	15



1 | Goal

In this laboratory you will learn **Object-Oriented Programming (OOP)** in Rust by building a terminal-based game: “**Human vs. Zombie**”. The game simulates a turn-based showdown between humans and zombies on a 2D grid. You will design structs, implement their behaviour, and compose them into a working game.

The lab has **two parts**:

- **Part 1**: Build the game using traditional **OOP** with separate vectors for humans and zombies.
- **Part 2**: Refactor to use an **Entity** trait for polymorphism, learning how traits enable dynamic dispatch.

At the end of this lab, you will have:

- A working terminal game rendered with **American Standard Code for Information Interchange (ASCII)** characters
- Hands-on experience with Rust structs, **impl** blocks and encapsulation
- An understanding of getter/setter patterns
- A **World** that orchestrates movement, infection, and decay
- Experience with traits and **Box<dyn Trait>** for polymorphism

1.1 The Game

The game takes place on a rectangular grid. Each cell can be empty, occupied by a human **H**, or occupied by a zombie **Z**.

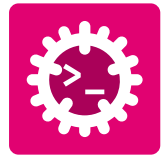
- **Humans** flee from zombies and try to survive.
- **Zombies** chase humans, infect them on contact, and decay over time.
- The game runs in **ticks**: movement $\Rightarrow$ infection $\Rightarrow$ decay $\Rightarrow$ cleanup.
- The game ends when all humans are eliminated (zombies win) or all zombies decay (humans win).

Humans H	Moves away from the nearest zombie
Zombies Z	Moves toward the nearest human
Infection	A human adjacent to ≥ 1 zombie turns into a zombie
Decay	Zombies lose 10 HP per tick and eventually die
Win condition	All zombies dead $\Rightarrow$ humans win. All humans infected $\Rightarrow$ zombies win

Table 1 - Game rules summary

1.2 Project Setup

The files for this lab are located in the **aprog-labs** repository in the **labs/02-oop/01-hvz** directory.



2 | The Position Struct

Open `src/position.rs`. Unlike the other modules, the **Position** struct is **not** pre-written - you must write it entirely from scratch, including the struct definition, the required trait derives, and all methods.

 Position
□ <code>x : i32</code> □ <code>y : i32</code>
● <code>new(x, y)</code> ● <code>x()</code> ● <code>y()</code> ● <code>set_x(x)</code> ● <code>set_y(y)</code> ● <code>distance(other)</code>

Figure 1 - The **Position** struct represents a point in 2D space with **x** and **y** coordinates.

What you need to write:

1. The public struct **Position** with two private fields **x** and **y** (both **i32**).
2. The required trait derives - the provided game code compares positions (**==**) and copies them. The compiler can help to figure out the missing traits.
3. An **impl Position** block with six methods:
 - **new(x, y)** - Constructor that creates a **Position**.
 - **x()** - Getter for the **x** coordinate.
 - **y()** - Getter for the **y** coordinate.
 - **set_x(x)** - Setter for the **x** coordinate.
 - **set_y(y)** - Setter for the **y** coordinate.
 - **distance(other)** - Returns the **Euclidean distance** to another position.

The distance formula:

$$d = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$



- Write the struct definition, add the required derives, and implement all six methods.
- Run **just test**: the 6 tests under **position::** should pass.



3 | The Human Struct

A **Human** is a minimal entity that only holds a position on the grid. It has no health or attack - survival is handled by the infection rules, not individual stats.

Open `src/human.rs`. The movement logic (`calculate_new_position()` and `nearest_zombie()`) is already implemented for you.

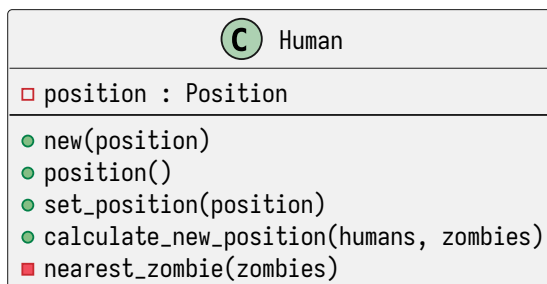


Figure 2 - The **Human** struct represents a human entity with a position on the grid.

You only need to fill the `todo!()` stubs:

- `new(position)` - Creates a human at the given position.
- `position()` - Returns a reference to the human's position.
- `set_position(position)` - Updates the human's position.



- Fill each `todo!()` in `impl Human`.
- **Question:** Why does `position()` return `&Position` instead of `Position`?
- Run `just test`: the 6 tests under `human::` should pass.

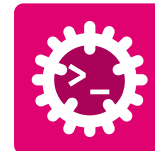
3.1 How Movement Works (Pre-implemented)

The `calculate_new_position()` method is already written. It calls `nearest_zombie()` (also pre-implemented) to find the closest zombie, then computes a direction vector and asks `World::find_free_neighbor_static` for a valid empty cell.

You do **not** need to modify these methods.



Read and understand the implementation in `calculate_new_position()` and `nearest_zombie()`.



4 | The Zombie Struct

A **Zombie** has a position and a **stamina** value. Stamina decreases by **ZOMBIE_DECAY** (10) each tick - when it reaches 0 the zombie dies.

 Zombie
□ position □ stamina
○ ZOMBIE_STAMINA : u32 = 100
● new(position) ● position() ● set_position(position) ● stamina() ● set_stamina(stamina) ● is_alive() ● calculate_new_position(humans, zombies) ■ nearest_human(humans)

Figure 3 - The **Zombie** struct represents a zombie entity with a position and stamina.

Open **src/zombie.rs**. The movement logic (**calculate_new_position()** and **nearest_human()**) is already implemented for you. You only need to fill the **todo!()** stubs:

- **new(position)** - Creates a new zombie (stamina starts at **ZOMBIE_STAMINA**).
- **position()** - Returns a reference to the zombie's position.
- **set_position(position)** - Updates the zombie's position.
- **stamina()** - Returns the current stamina value.
- **set_stamina(stamina)** - Updates the stamina value.
- **is_alive()** - Returns **true** if stamina > 0.



- Fill each **todo!()** in **impl Zombie**.
- Run **just test**: the 10 tests under **zombie::** should pass.

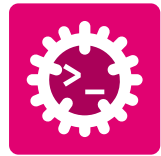
4.1 How Movement Works (Pre-implemented)

The **calculate_new_position()** method is already written. It calls **nearest_human()** to find the closest human, then moves **two** cells toward it (zombies are faster than humans). It calls **World::find_free_neighbor_static** twice - once for the first step, once for the second - and falls back to the first step if the second is blocked.

You do **not** need to modify these methods.



- Read and understand the implementation in **calculate_new_position()** and **nearest_human()**.



5 | The World Struct

Open `src/world.rs`. The `World` struct and most methods are already implemented for you. You only need to implement two `todo!()` stub:

- `new(width, height)` - Creates an empty world with the given dimensions.
- `tick_decay()` - Zombies lose `ZOMBIE_DECAY` stamina per tick. Dead zombies are removed.



- Implement `new()` and `tick_decay()` in `src/world.rs`.
- Run `just test: test_01_new, test_10`, and `test_11` should pass.

5.1 Pre-implemented Methods

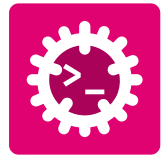
The following methods are already written. You do **not** need to modify them, but you should read through them to understand how the game works:

- `add_human(human)` / `add_zombie(zombie)` - Add entities to the world.
- `tick_movement()` - For each entity, computes a new position and moves it immediately before processing the next one, preventing two entities from landing on the same cell.
- `tick_infection()` - Humans adjacent to a zombie are converted into zombies (two-pass snapshot approach).
- `process_turn()` - Runs all three phases in order.
- `find_free_neighbor_static(pos, dx, dy, humans, zombies)` - Finds an empty adjacent cell, trying the preferred direction first then falling back to alternatives.
- `display()` / `run()` - Renders the game in the terminal.

5.2 Final Verification



- Run `just test` - all 12 world tests should pass.
- Run `just run` - the game should render with **bold green H** and **bold red Z** characters.
- A winner should be declared (humans or zombies).



6 | Checkout Part 1

Before moving to Part 2, ensure you have completed:

Position struct (written from scratch)

- ☐ Struct with **x**, **y** fields and required derives
- ☐ **new()**, **x()**, **y()**, **set_x()**, **set_y()** implemented
- ☐ **distance()** computes Euclidean distance correctly
- ☐ 6 unit tests pass

Human struct

- ☐ **new()** sets the initial position
- ☐ **position()** returns **&Position**
- ☐ **set_position()** updates the position
- ☐ 6 tests pass (including movement tests)

Zombie struct

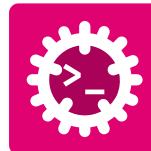
- ☐ **new()** creates zombie with **ZOMBIE_STAMINA**
- ☐ **position()** and **set_position()** implemented
- ☐ **stamina()** and **set_stamina()** implemented
- ☐ **is_alive()** returns **stamina > 0**
- ☐ 9 tests pass (including movement tests)

World struct

- ☐ **new()** constructor implemented
- ☐ **tick_decay()** implemented
- ☐ All 12 world tests pass

Final verification

- ☐ **just test** reports all tests passing
- ☐ **just run** shows the game running in the terminal
- ☐ A winner is declared (humans or zombies)



7 | The Entity Trait

In Part 1, **World** kept humans and zombies in two separate vectors and had to call different methods on each.

```
pub struct World {
    ...
    pub humans: Vec<Human>,
    pub zombies: Vec<Zombie>,
    ...
}
```

Every time you added a new creature type, you would need another vector, another **add_*** method, and another branch in every loop.

The goal of Part 2 is to introduce a common **Entity** trait so that **World** can store all creatures in a single **Vec<Box<dyn Entity>>** and treat them uniformly.

7.1 Step 1: Define the Trait

Create **src/entity.rs** with a trait that describes what every entity can do:

```
use crate::position::Position;
pub trait Entity {
    fn position(&self) -> &Position;
    fn set_position(&mut self, pos: Position);
    fn calculate_new_position(&self, others: &[Box<dyn Entity>]) -> Option<Position>;
    fn tick(&mut self) {}
    fn is_alive(&self) -> bool;
    fn is_human(&self) -> bool;
    fn as_str(&self) -> &str;
}
```

Add **pub mod entity;** to **src/lib.rs**.



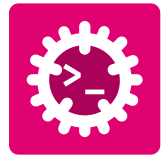
- Create **src/entity.rs** with the trait above.
- Add **pub mod entity;** to **src/lib.rs**.

7.2 Step 2: Implement Entity for Human

In **src/human.rs**, replace the standalone methods with an **impl Entity for Human** block. The key change: function signatures now use **&[Box<dyn Entity>]** instead of concrete types.



- Implement **Entity** for **Human**.
- Update **nearest_zombie()** to take **&[Box<dyn Entity>]** and filter with **entity.is_human()**.
- Run **just test:** the 6 **human::** tests should pass.



7.3 Step 3: Implement Entity for Zombie

Same pattern for `src/zombie.rs`.



- Implement **Entity** for **Zombie**.
- Update `nearest_human` to take `&[Box<dyn Entity>]`.
- Run `just test`: the 10 `zombie::` tests should pass.

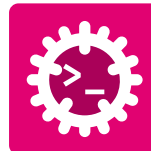
7.4 Updating `find_free_neighbor`

Both **Human** and **Zombie** call `find_free_neighbor_static()` in their `calculate_new_position()`.



In `src/utils.rs`.

- Remove the Part 1 version of `find_free_neighbor_static()`.
- Uncomment the Part 2 version of `find_free_neighbor()` (which takes `&[Box<dyn Entity>]`).
- Update the calls in `human.rs` and `zombie.rs`.



8 | World Refactoring

Now that **Human** and **Zombie** both implement **Entity**, refactor **World** to use a single vector instead of two.

8.1 The Struct

Replace **humans** and **zombies** with a single **entities** field:

```
pub struct World {  
    pub width: u32,  
    pub height: u32,  
    entities: Vec<Box<dyn Entity>>,  
    pub turn: u32,  
}
```

8.2 Adding Entities

The internal storage changes to **Vec<Box<dyn Entity>>**, but the public API stays the same so **main.rs** does not need to change:

```
pub fn add_human(&mut self, human: Human) {  
    self.entities.push(Box::new(human));  
}  
  
pub fn add_zombie(&mut self, zombie: Zombie) {  
    self.entities.push(Box::new(zombie));  
}
```



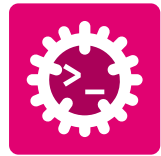
- Update **World** struct and adapt **add_human/add_zombie** to wrap in **Box::new()**.
- Run **just test**: it should compile.

8.3 Movement

Merge the two separate loops (zombies, then humans) into one loop over **self.entities**. Each entity now calls **calculate_new_position(&self.entities)** through the trait.



We compute all new positions first (immutable borrow), then apply them in a second loop (mutable borrow). This avoids borrow-checker conflicts without cloning.



8.4 Infection

Same two-pass snapshot approach, but now checking `entity.is_human()` instead of iterating `self.humans`:

```
fn tick_infection(&mut self) {
    let mut to_remove = Vec::new();
    let mut new_zombies = Vec::new();

    for (i, entity) in self.entities.iter().enumerate() {
        if !entity.is_human() {
            continue;
        }
        let pos = entity.position();
        let nearby_zombies = self.entities.iter().filter(|e| {
            if e.is_human() {
                return false;
            }
            let dx = (e.position().x() - pos.x()).abs();
            let dy = (e.position().y() - pos.y()).abs();
            dx <= 1 && dy <= 1
        }).count();
        if nearby_zombies >= INFECTION_THRESHOLD {
            to_remove.push(i);
            new_zombies.push(crate::zombie::Zombie::new(*pos));
        }
    }

    for &i in to_remove.iter().rev() {
        self.entities.swap_remove(i);
    }
    for zombie in new_zombies {
        self.entities.push(Box::new(zombie));
    }
}
```



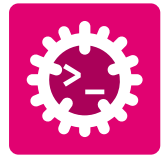
- Copy-paste the `tick_infection` above into `src/world.rs`.
- Run **just test**: the infection tests should pass.

8.5 Decay

Call `entity.tick()` on all entities, then retain only alive ones.



- Refactor `tick_decay()` to loop over `self.entities`.
- Run **just test**: the decay tests should pass.



8.6 Display

`count_humans()` and `count_zombies()` already exist from Part 1. Update them to iterate over `self.entities` using `is_human()`. Display now calls `entity.as_str()` instead of hardcoding colors.

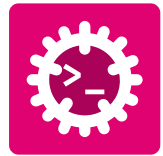


- Refactor all remaining methods in `src/world.rs`.
- Run **just test**: all 8 world tests should pass.
- Run **just run**: the game should work correctly.

8.7 Verification



- All tests pass (**just test**).
- Game runs (**just run**).
- No `nearest_human()` or `nearest_zombie()` on `World`.
- Commit and push.



9 | Checkout Part 2

Before finishing, ensure you have completed:

Entity trait

- ☐ `src/entity.rs` created with all methods
- ☐ `pub mod entity;` added to `src/lib.rs`

Human implements Entity

- ☐ `nearest_zombie` takes `&[Box<dyn Entity>]`
- ☐ `calculate_new_position` takes `&[Box<dyn Entity>]`
- ☐ 6 human tests pass

Zombie implements Entity

- ☐ `nearest_human` takes `&[Box<dyn Entity>]`
- ☐ `calculate_new_position` takes `&[Box<dyn Entity>]`
- ☐ `tick()` decreases stamina
- ☐ 10 zombie tests pass

find_free_neighbor

- ☐ Moved to `src/utils.rs`
- ☐ Part 2 version uncommented, Part 1 removed
- ☐ Calls updated in `human.rs` and `zombie.rs`

World refactored

- ☐ `entities: Vec<Box<dyn Entity>>`
- ☐ `add_human` / `add_zombie` wrap in `Box::new()`
- ☐ `tick_movement` uses one loop over entities
- ☐ `tick_infection` checks `is_human()`
- ☐ `tick_decay` calls `entity.tick()` and retains alive
- ☐ `count_humans` / `count_zombies` iterate entities
- ☐ `display` uses `entity.as_str()`
- ☐ All 8 world tests pass

Final verification

- ☐ `just test` reports all tests passing
- ☐ `just run` shows the game running in the terminal
- ☐ No `nearest_human()` or `nearest_zombie()` on `World`



Glossary

ASCII – American Standard Code for Information Interchange: Character encoding standard for representing text. [3](#)

OOP – Object-Oriented Programming: Programming paradigm based on abstraction, encapsulation, modularity and hierarchy. [3](#), [3](#)